

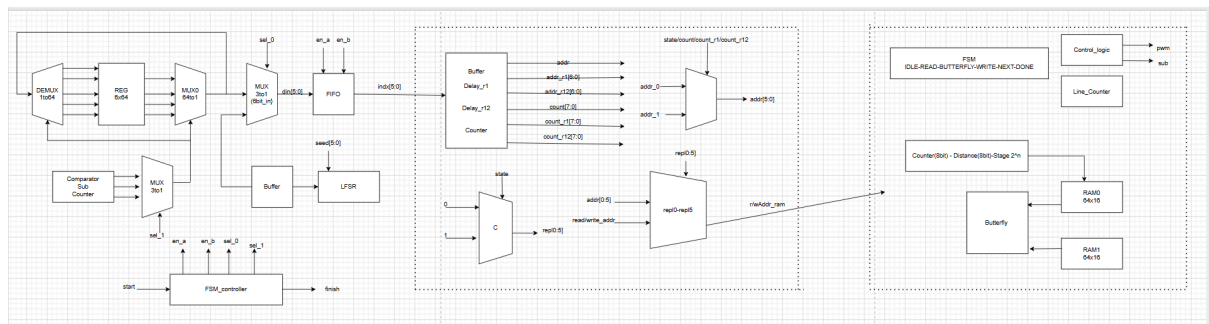
I. Cơ sở lý thuyết:

- Bài báo A Hardware-Friendly Shuffling Countermeasure Against Side-Channel Attacks for Kyber đề xuất một cơ chế SCA (Side-Channel Attack – SCA) cho thuật toán

CRYSTALS-Kyber theo hướng “hardware-friendly”, tức là phù hợp để triển khai trên phần cứng FPGA với chi phí tài nguyên thấp.

- Ý tưởng chính của bài báo là không thay đổi dữ liệu mật hay thêm masking phức tạp, mà thay đổi thứ tự truy cập bộ nhớ trong quá trình thực hiện NTT bằng cách sử dụng một bộ sinh hoán vị ngẫu nhiên gọi là Random Permutation Generator (RPG). Thay vì đọc dữ liệu tuần tự theo địa chỉ cố định như 0,1,2,3..., hệ thống sẽ truy cập RAM theo thứ tự ngẫu nhiên như 11,37,28,35..., khiến switching activity của mạch trở nên khó dự đoán hơn và làm giảm khả năng phân tích power trace của kẻ tấn công. RPG trong bài báo gồm các khối chính như LFSR để tạo số ngẫu nhiên, FIFO để lưu chỉ số tạm thời, REG để chứa permutation, cùng controller điều khiển quá trình khởi tạo và xáo trộn địa chỉ. Quá trình tạo permutation được chia thành nhiều giai đoạn như initialization, shuffling và cyclic shift nhằm tạo ra tập địa chỉ truy cập ngẫu nhiên hoàn chỉnh. Kết quả của bài báo cho thấy phương pháp này giúp giảm leakage hiệu quả trong khi mức tăng tài nguyên phần cứng và độ trễ là khá nhỏ, phù hợp cho các thiết kế Kyber trên FPGA hoặc ASIC.

II. Mô hình thiết kế:



1. Random permutation generate -RPG:

- Bắt đầu từ khối LFSR, nơi liên tục sinh ra các bit ngẫu nhiên dựa trên giá trị seed ban đầu. Các bit ngẫu nhiên này được đưa vào khối buffer để gom nhiều bit 1-bit thành một chỉ số ngẫu nhiên 6-bit (rand_idx). Sau mỗi sáu chu kỳ clock, buffer xuất ra một giá trị ngẫu nhiên hoàn chỉnh và phát tín hiệu valid để báo dữ liệu hợp lệ.
- Chỉ số ngẫu nhiên sau đó được đưa qua MUX1 trước khi ghi vào FIFO. Tùy theo tín hiệu điều khiển sel0, dữ liệu đưa vào FIFO có thể là giá trị ngẫu nhiên mới từ LFSR hoặc là giá trị permutation đã được lưu trước đó trong bộ nhớ REG. FIFO hoạt động ở hai chế độ: chế độ input mode để nhận dữ liệu mới từ LFSR hoặc REG, và chế độ cyclic mode để tuần hoàn lại dữ liệu bên trong FIFO nhằm giảm tài nguyên phần cứng. Hai tín hiệu ena và enb được controller sử dụng để điều khiển hai chế độ hoạt động này.

- Dữ liệu đầu ra của FIFO được chuyển đến khối `index_control`, nơi các chỉ số ngẫu nhiên được điều chỉnh để phù hợp với hai giai đoạn shuffling của kiến trúc. Khối này tạo ra giá trị `idx`, chính là địa chỉ hoán vị cuối cùng được dùng để truy cập bộ nhớ permutation trong phần REG của RPG. REG lưu toàn bộ bảng hoán vị và sử dụng MUX0 cùng DEMUX để đọc và cập nhật các phần tử permutation tương ứng với `idx`

2. Address Controller :

- Khối Address Controller có nhiệm vụ chuyển đổi các địa chỉ tuần tự do NTT_core tạo ra thành các địa chỉ đã được xáo trộn bởi RPG. Đường đi dữ liệu bắt đầu từ đầu vào `addr_in`, là giá trị permutation (`idx_prime`) do RPG cung cấp. Giá trị này được đưa vào phần B của Address Controller để tạo ra các phiên bản delay khác nhau như `addr_r1` và `addr_r12` thông qua các shift register 1 chu kỳ và 12 chu kỳ. Đồng thời, một bộ đếm nội bộ cũng tạo ra các tín hiệu `count`, `count_r1`, và `count_r12` nhằm đồng bộ hóa các stage xử lý của NTT.
- Trong phần D, các giá trị permutation được ghép thêm bit hoặc kết hợp với tín hiệu line để tạo thành sáu địa chỉ xáo trộn mới `addr0` đến `addr5`
- Song song với đó, phần C của Address Controller sinh các tín hiệu điều khiển `repl0` đến `repl5` dựa trên trạng thái hoạt động của NTT như `pwm` và `sub`. Các tín hiệu này quyết định thời điểm cần bật hoặc tắt cơ chế xáo trộn địa chỉ.
- Cuối cùng, trong phần A, sáu bộ MUX 2-to-1 được sử dụng để lựa chọn giữa địa chỉ gốc từ NTT_core và địa chỉ đã được xáo trộn từ `addr0~addr5`

3. NTT_core:

-Ý tưởng : Khối **NTT_core** được thiết kế nhằm mô phỏng datapath của thuật toán Number Theoretic Transform (NTT) trong Kyber, đồng thời tích hợp cơ chế shuffling địa chỉ để phục vụ Side-Channel Attack – SCA.

-Mục tiêu chính của thiết kế không phải tối ưu hiệu năng tính toán NTT hoàn chỉnh mà tập trung vào:

- tạo mẫu truy cập bộ nhớ giống NTT thực,
- kết hợp cơ chế random permutation,
- mô phỏng leakage trên đường truy cập RAM

-Cách hoạt động :

FSM trong NTT_core tạo ra: `raddr_RAM0` và `raddr_RAM1` -đây là các địa chỉ truy cập tuần tự của butterfly trong NTT.

Các địa chỉ gốc không được dùng trực tiếp để truy cập RAM.

Thay vào đó:

- Address Controller nhận `idx_prime`
- rồi ánh xạ lại thành: `raddr_p_RAM0` và `raddr_p_RAM1` là các địa chỉ đã được xáo trộn.

Sau khi có địa chỉ shuffled:

`A <= RAM0[raddr_p_RAM0];`

`B <= RAM0[raddr_p_RAM1];`

hai toán hạng butterfly được lấy từ RAM.

Thiết kế hiện tại dùng butterfly đơn giản nhằm:

- giảm độ phức tạp,
- tập trung vào memory access leakage

4 .Trace Power

Trong thiết kế này, khối `power_trace` được sử dụng để mô phỏng rò rỉ công suất (power leakage) của phân cứng

Mô hình này giả sử: Power n số bit thay đổi giữa hai trạng thái liên tiếp

Code:

`else if(state == READ)`

vì:

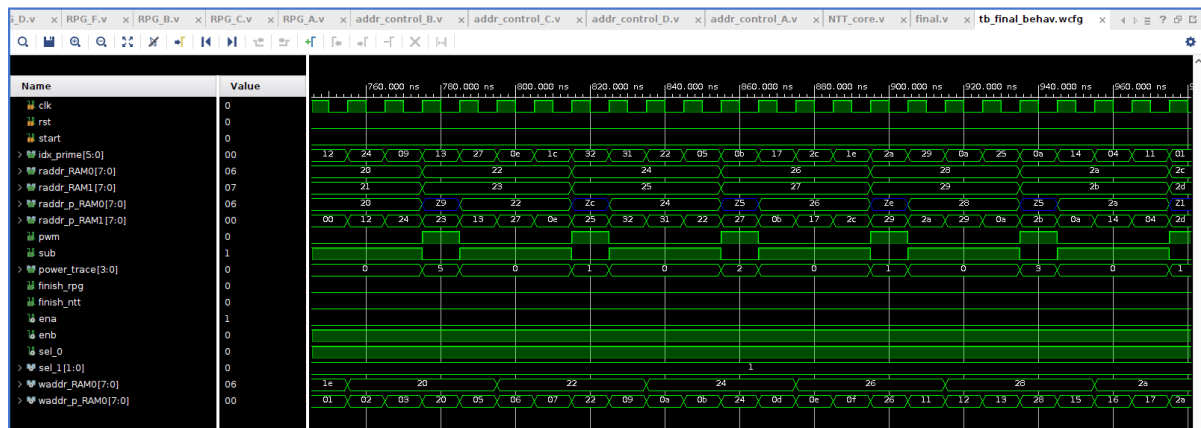
- leakage mạnh nhất thường xảy ra lúc truy cập RAM
- memory decoder switching.

Do đó:

- chỉ trace ở READ stage để mô phỏng realistic hơn.

III. Kết quả

1 Waveform :



-Giải thích

idx_prime[5:0] thay đổi liên tục: 12 → 24 → 09 → 13 → 27 → ...

Đây là:

- chỉ số permutation được sinh bởi RPG,
- dùng để random hóa địa chỉ truy cập RAM.

Các tín hiệu:raddr_p_RAM0 và raddr_p_RAM1 đã được ánh xạ lại bởi Address Controller.

pwm = 1 xuất hiện trong:READ stage.

sub = 1 xuất hiện trong: BUTTERFLY stage.

power_trace[3:0] : XOR địa chỉ shuffled hiện tại và địa chỉ shuffled trước đó.

Giải sử:

current=20 = 0010_0000

previous= 29 = 0010_1001

XOR: 0000_1001

Có: 2 bit đổi trạng thái → switching activity tăng.

2.Tài nguyên sử dụng

Name	^1	Slice LUTs (17600)	Slice Registers (35200)	F7 Muxes (8800)	F8 Muxes (4400)	Bonded IOB (54)	BUFGCTRL (32)
final		453	877	91	43	49	1
ADDR0 (addres_controller)		0	6	0	0	0	0
NTT0 (NTT_core)		49	48	0	0	0	0
RPG0 (top)		404	823	91	43	0	0

-Schematic

